

# PRAKTIKUM SINGLE LINKED LIST

## A. KONSEP LINEAR LIST

List linier adalah sekumpulan elemen *bertipe* sama yang saling terkait dengan elemen selanjutnya dan mempunyai keterurutan tertentu. Setiap elemen pada List Linear terdiri dari dua bagian, yaitu: informasi mengenai nilai elemennya, dan informasi mengenai alamat elemen suksesornya atau alamat elemen selanjutnya.

Spesifikasi untuk tipe list linear adalah:

*type* ElmtList : <info : InfoType, Next : address>

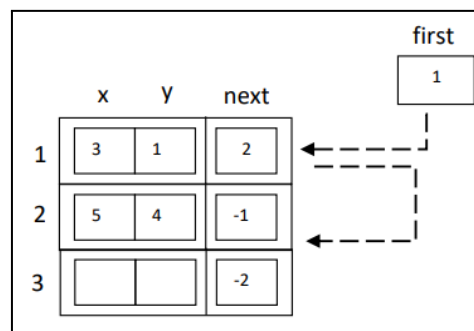
InfoType adalah sebuah *type* terdefinisi yang menyimpan informasi sebuah elemen list.

Next adalah address ("alamat") dari elemen berikutnya (suksesor).

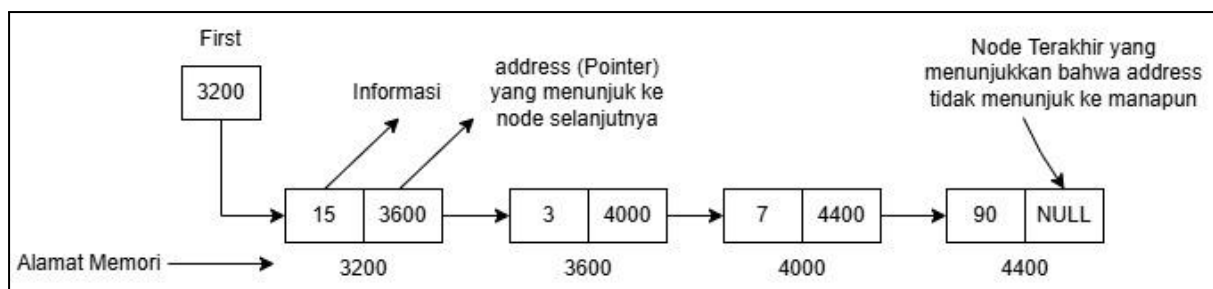
Dengan demikian, jika didefinisikan First adalah alamat elemen pertama list, maka elemen berikutnya dapat diakses secara berurutan dari field Next elemen tersebut.

Address sebuah nilai yang menunjukkan "alamat" elemen list maka disebut sebagai pengalokasian elemen. Sebaliknya jika address bernilai suatu konstanta Nil atau berisi nilai yang tidak menunjuk ke alamat, maka tidak menunjuk ke elemen list atau alamat tidak terdefinisi. Alamat ini nantinya akan didefinisikan secara lebih konkret ketika list linier diimplementasi pada struktur data fisik.

Linear linear list dapat dialokasikan dalam bentuk statis (alamat berisi nilai integer sbg nomor alamat element list) dan bentuk dinamis (alamat bertipe pointer yang berisi nilai "alamat" di memori).



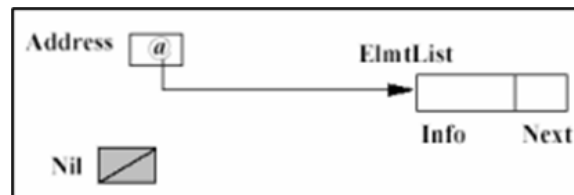
Representasi Linear List Alokasi Statis



Representasi Linear List Alokasi Dinamis

## B. LINKED LIST

Pada kegiatan praktikum hari ini membahas linear list menggunakan alokasi dinamis atau sering disebut sebagai linked list. Atribut ADT Linked List



Jadi, sebuah list linier dikenali dengan :

- **Elemen pertamanya**, biasanya melalui alamat elemen pertama yang disebut : **First**;
- Alamat **elemen berikutnya** (suksesor) diakses melalui informasi **next**;
- Setiap elemen mempunyai alamat, yaitu tempat elemen disimpan yang dapat ditunjuk oleh address. Agar sebuah elemen dapat ditunjuk oleh address, maka harus mendefinisikan alamat pada elemen tersebut. Dengan menggunakan alamat tersebut, informasi yang tersimpan pada elemen list dapat diakses;
- **Elemen terakhir** adalah elemen yang berada pada paling belakang.

Beberapa Definisi :

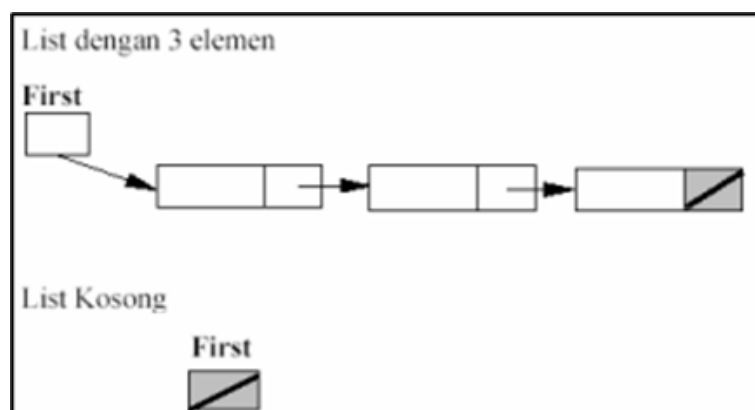
Jika L adalah list dan P adalah address maka dapat diasumsikan bahwa:

- Alamat elemen pertama list L dapat ditunjuk dengan notasi : **First(L)**
- Elemen yang diacu oleh P dapat dikonsultasi informasinya dengan notasi Selektor :

**Info(P)**, untuk menunjuk elemen info

**Next(P)**, untuk menunjuk elemen next (elemen selanjutnya)

- List L adalah list kosong, jika **First(L) = nil**
- Elemen terakhir dikenali, misal jika Last adalah alamat elemen terakhir, maka **Next(Last) = nil**



## Operasi Linked List:

### 1). INSERT (Melakukan Penyisipan pada List)

Jika sesuatu yang disisipkan ke list berupa address (sudah dialokasi sebelumnya) maka proses penyisipan dapat langsung disisipkan. Tetapi jika sesuatu yang disisipkan adalah sebuah nilai maka dilakukan proses pengalokasian terlebih dahulu. Alokasi adalah pembentukan sebuah address yang mengacu/menunjuk ke elemen list. Alokasi dilakukan dengan;

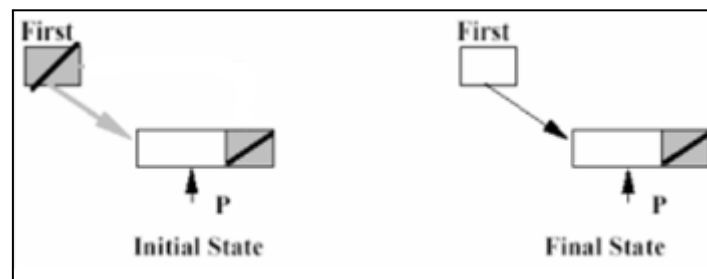
1. Buat sebuah elemen list
2. Buat address (P) untuk menunjuk ke elemen list tersebut
3. Jika alokasi tidak berhasil maka P diisi nil
4. Jika alokasi berhasil, maka isi komponen element list pada address (P) : Info = nilai dan Next = nil

Proses *insertion* saat memasukkan element node ke list ditunjukkan dengan tanda panah berwarna abu-abu pada setiap visualisasi dalam Gambar Initial State operasi *insert* di ketiga posisi: *first*, *middle* dan *delete*.

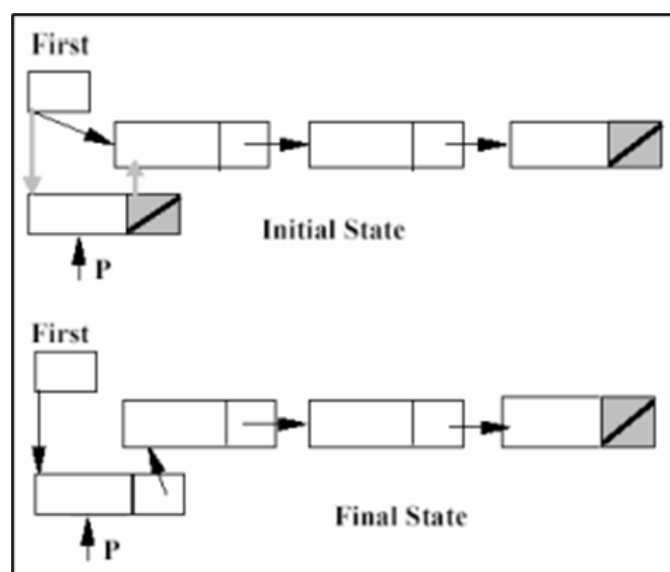
#### INSERT-First (diketahui alamat)

Menambahkan sebuah elemen yang diketahui alamatnya sebagai elemen pertama list. Ada dua kemungkinan yaitu kondisi list kosong dan tidak kosong.

Insert elemen pertama pada list kosong :



Insert elemen pertama pada list tidak kosong :



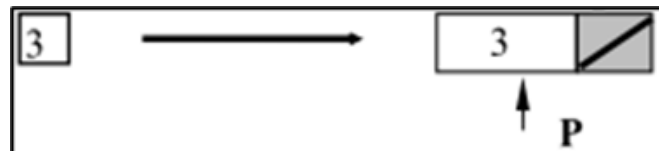
### INSERT-First (diketahui nilai)

Menambahkan sebuah elemen yang diketahui nilainya sebagai elemen pertama list. Ada dua kemungkinan yaitu kondisi list kosong dan tidak kosong.

#### Tahap pertama :

Jika diketahui nilai yang akan diinsertkan adalah 3 sebagai elemen pertama List. Oleh karena yang diketahui adalah nilai, maka harus dialokasikan dahulu sebuah elemen supaya nilai 3 dapat di-insert.

Jika alokasi berhasil, maka P tidak sama dengan Nil

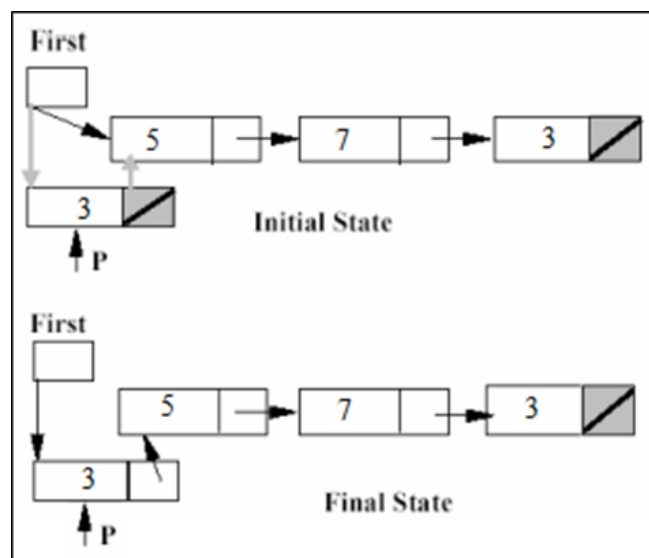


#### Tahap kedua : insert

Insert elemen pertama pada list kosong :

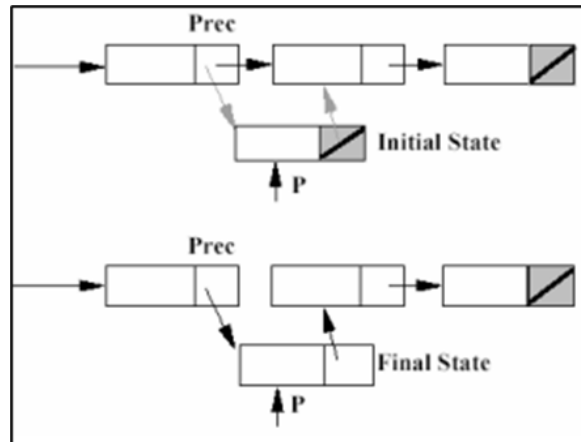


Insert elemen pertama pada list tidak kosong :



### INSERT-After :

Menyisipkan sebuah elemen beralamat P setelah suksesor dari sebuah elemen list linier yang beralamat Prec. Jika yang disisipkan adalah sebuah nilai, maka lakukan terdahulu alokasi untuk mendapatkan alamat elemen tersebut (proses alokasi sudah disebutkan diatas).

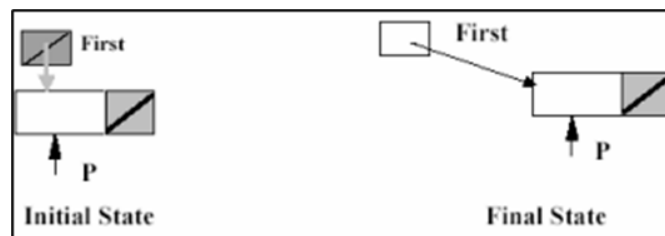


### INSERT-Last

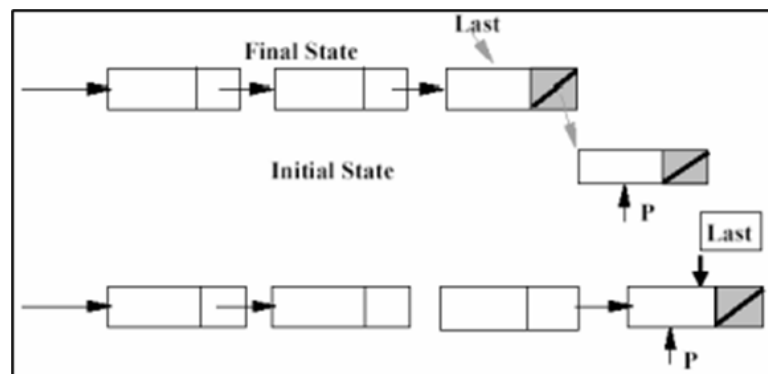
Menyisipkan sebuah elemen beralamat P setelah sebagai elemen terakhir sebuah list linier. Jika yang disisipkan adalah sebuah nilai, maka lakukan terdahulu alokasi untuk mendapatkan alamat elemen tersebut (proses alokasi sudah disebutkan diatas).

Ada dua kemungkinan sebelum penyisipan elemen yaitu kondisi list kosong dan tidak kosong.

Insert sebagai elemen terakhir pada list kosong :



Insert sebagai elemen terakhir pada list tidak kosong :



## 2). DELETE (Melakukan Penghapusan elemen pada List)

Setelah penghapusan, elemen yang dihapus dicatat alamatnya dan kadang kala alamat elemen yang tersebut akan didealokasi. Dealokasi adalah proses pembebasan address (P) sehingga tidak menunjuk ke elemen list.

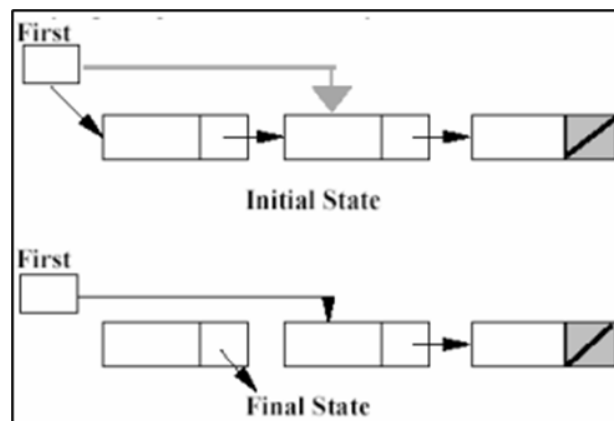
Sebagai catatan bahwa :

- Operasi delete yang tidak melakukan dealokasi address, dilakukan jika address tersebut masih dibutuhkan. Misalnya elemen yang dihapus akan dijadikan anggota elemen list lain: dalam hal ini tidak perlu melakukan dealokasi, kemudian alokasi ulang ketika ditambahkan ke list yang baru. Beberapa list dengan elemen sejenis seringkali harus dikelola dalam program; misalnya list mahasiswa per matakuliah.
- Operasi yang sekaligus melakukan dealokasi address, biasanya dilakukan kepada pengguna yang tidak perlu mengetahui “keberadaan” address dan yang hanya perlu memanipulasi informasi pada field info.

Proses *deletion* saat menghapus element node dalam list ditunjukkan dengan tanda panah berwarna abu-abu pada setiap visualisasi dalam Gambar Initial State operasi *delete* di ketiga posisi: *first*, *middle* dan *delete*.

### DELETE-First

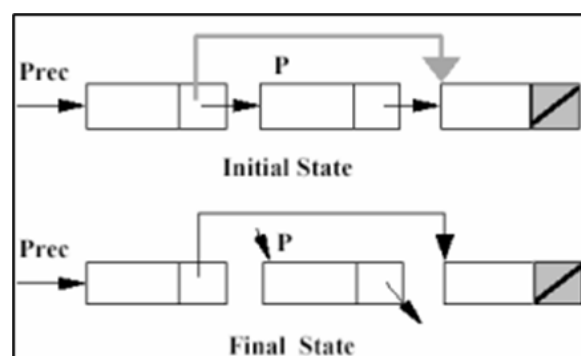
Menghapus elemen pertama list linier. Untuk proses penghapusan harus dipastikan bahwa element pada list tidak kosong.



### DELETE-After

Menghapus elemen yang berada ditengah list linier. Untuk proses penghapusan harus dipastikan bahwa element pada list tidak kosong dan jumlahnya lebih dari 1.

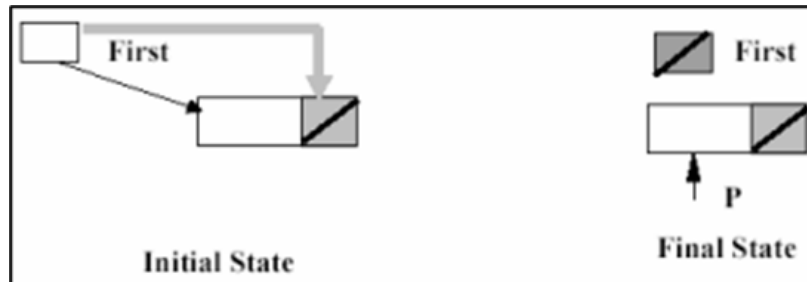
Penghapusan suksesor sebuah elemen :



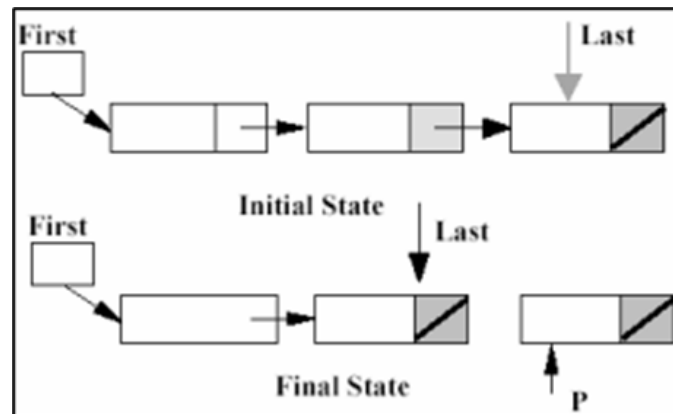
### DELETE-Last

Menghapus elemen terakhir list dapat dilakukan jika alamat dari elemen sebelum elemen terakhir diketahui. Untuk proses penghapusan harus dipastikan bahwa element pada list tidak kosong. Persoalan selanjutnya menjadi persoalan DELETE AFTER, jika Last bukan satu-satunya elemen list linier. Ada dua kemungkinan setelah proses penghapusan yaitu kondisi list menjadi kosong atau tidak.

Kasus list menjadi kosong setelah penghapusan :



List tidak menjadi kosong (masih mengandung elemen) setelah penghapusan :



### C. STUDI KASUS

Manfaatkan ADT Linked List dengan membuat main program sesuai dengan mengikuti spesifikasi program pembuatan deret bilangan ganjil dengan langkah berikut:

- 1) Create List untuk deret bilangan  
> **F.S.** : List terbuat dengan anggota elemen kosong = {}
- 2) masukan bilangan 7 sebagai elemen pertama {*insert first*}  
> **F.S.** : elemen List = {7}
- 3) masukan bilangan 11 sebagai elemen kedua {*insert last*}  
> **F.S.** : elemen List = {7, 11}
- 4) masukan bilangan 9 diantara bilangan 7 dan 11 {*insert after*}  
> **F.S.** : elemen List = {7, 9, 11}
- 5) masukan bilangan 5 di awal list {*insert first*}  
> **F.S.** : elemen List = {5, 7, 9, 11}
- 6) masukan bilangan 13 di akhir list {*insert last*}  
> **F.S.** : elemen List = {5, 7, 9, 11, 13}
- 7) hapus elemen terakhir (bilangan 13) pada list {*delete last*}  
> **F.S.** : elemen List = {5, 7, 9, 11}
- 8) hapus bilangan 7 pada list  
> **F.S.** : elemen List = {5, 9, 11}
- 9) hapus elemen pertama (bilangan 5) pada list {*delete first*}  
> **F.S.** : elemen List = {9, 11}
- 10) hapus semua elemen list deret bilangan ganjil  
> **F.S.** : elemen List = {}

**Note:** agar perubahan operasi pada elemen list terlihat dan dinyatakan berhasil maka:

- a. tampilkan elemen datanya pada setiap instruksi dengan menambahkan sintaks print pada program setelah melakukan operasi.
- b. gambarkan representasi *linked list* pada setiap instruksi secara manual di dalam kertas.
- c. setiap kali akan melakukan penghapusan elemen pastikan terlebih dahulu bahwa info elemen yang akan dihapus tersedia pada list. Tolak penghapusan jika kondisi *linked list* dalam **underflow**, yaitu operasi DELETION tidak mungkin dilakukan lagi, karena list sudah kosong.
- d. setiap kali memasukan elemen ke dalam *linked list* pastikan bahwa hasil alokasi saat *create node* elemen list tidak null, artinya pengalokasian ke dalam memori berhasil. Namun, ketika **overflow** terjadi / hasil alokasi tidak berhasil mendapatkan alamat di dalam memori, maka proses *insertion* harus di tolak.